

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	22
REFERÊNCIAS.....	23

EMSE

O QUE VEM POR AÍ?

Você sabe qual é a utilidade dos algoritmos de otimização? No mundo da matemática, da ciência da computação e da engenharia, eles **assumem o papel de guias experientes na busca pela melhor solução possível.**

Seja para minimizar custos, maximizar lucros ou encontrar o caminho mais curto, esses algoritmos nos permitem navegar por complexas paisagens de possibilidades e alcançar resultados otimizados.

Nesta aula, você vai conhecer quais são os principais algoritmos de otimização no mundo de aprendizado de máquina e como utilizá-los ao seu favor para solucionar problemas complexos.

HANDS ON

Que tal aprender na prática como utilizar um algoritmo de otimização? Nesta aula você vai compreender o fascinante mundo da otimização, entendendo como utilizar algoritmos de gradiente para resolver problemas complexos e encontrar as melhores soluções.



SAIBA MAIS

Entendendo o gradiente descendente

O algoritmo do gradiente descendente é um algoritmo de otimização muito utilizado no aprendizado de máquina que é capaz de encontrar ótimas soluções quando precisamos otimizar um processo de convergência para encontrar o menor erro possível de um aprendizado de um algoritmo.

O grande objetivo dele é ajustar iterativamente os parâmetros do algoritmo para minimizar uma função de custo. Pareceu um pouquinho grego? Bem, eu vou te explicar com uma analogia bem mais simples!

Imagine-se em uma trilha desafiadora no topo de uma montanha, coberta por uma densa neblina. Você não consegue ver o caminho à frente, mas seus pés podem sentir a inclinação do solo sob você. Sua missão é chegar ao pé da montanha com segurança e rapidez. O gradiente descendente funciona de maneira similar a essa jornada aventureira.

Ele **guia seus modelos de aprendizado na direção ideal**, explorando o terreno montanhoso das funções matemáticas **até encontrar o caminho mais baixo, ou seja, a solução ideal**. O caminho com “menos obstáculos” nessa montanha e que te leva ao objetivo mais rápido parece ser o ideal, não? É assim que o gradiente funciona!

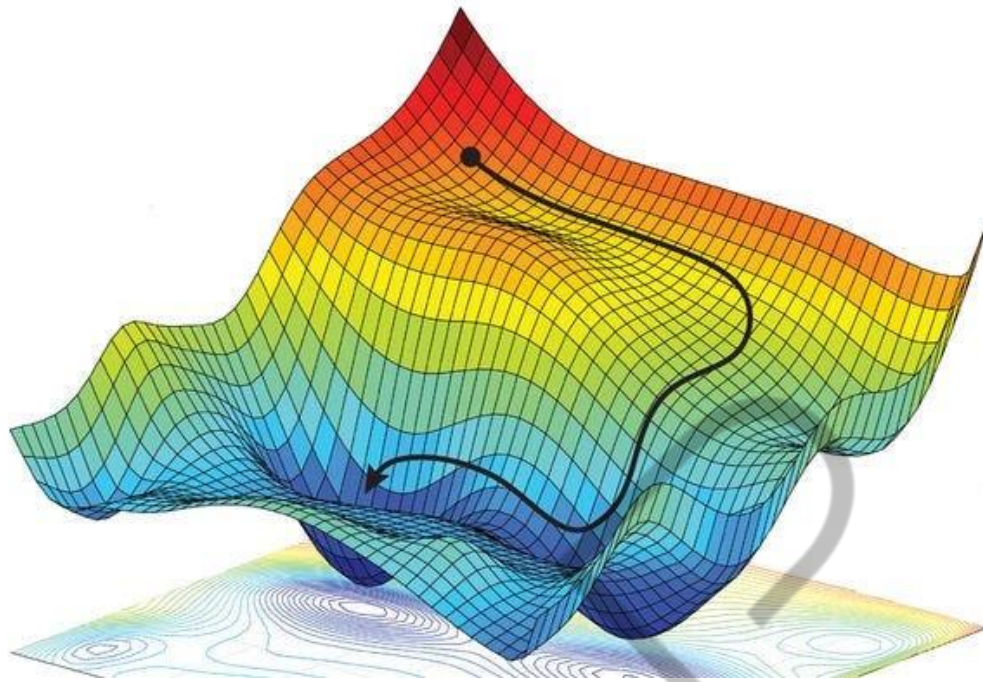


Figura 1 - A descida do gradiente
Fonte: Vaz (2020)

Observe a figura 2: temos em θ um valor que pode ser instanciado de forma aleatória (isso é chamado de inicialização aleatória) e então esse valor **será melhorado gradualmente**, dando pequenos passos, um de cada vez. Esses passos são realizados até que a função de custo representada no eixo y (cost) seja a menor possível (por exemplo: podemos pensar na métrica MSE). Esse passo será realizado até que o algoritmo convirja para um mínimo.

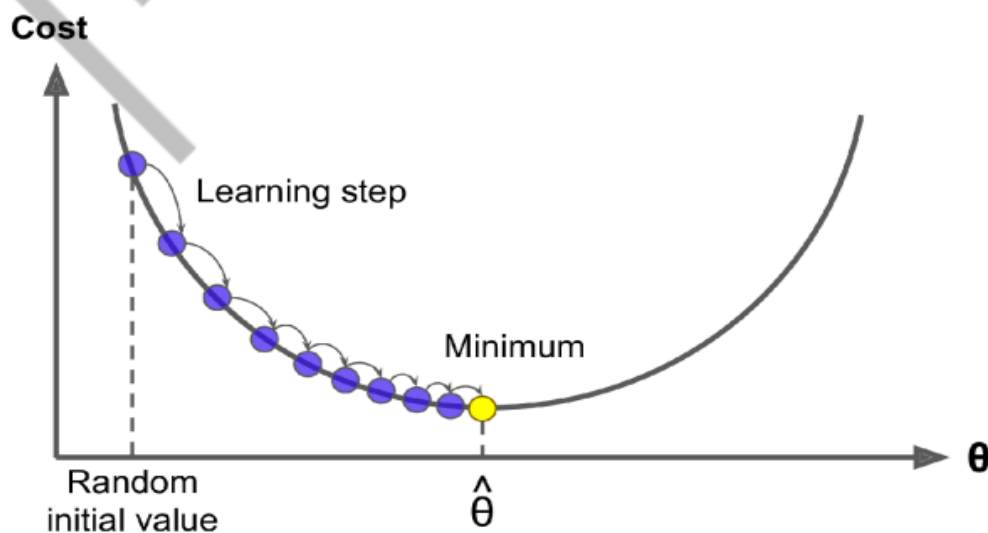


Figura 2 - Funcionamento do gradiente
Fonte: Géron (2019)

Esses passos que ocorrem durante a descida do gradiente são denominados de **taxa de aprendizagem**, também conhecida como learning rate. Podemos dizer que a learning rate controla o tamanho dos “passos” das interações para encontrar um melhor mínimo local (menor erro).

Uma taxa de aprendizagem baixa pode ser mais precisa, porém demorar para encontrar o mínimo local. Uma taxa de aprendizado alta pode deixar passar o melhor mínimo local. Assim, observe nas figuras a seguir, a diferença entre um valor de taxa de aprendizagem muito pequeno e uma com passos muito altos.

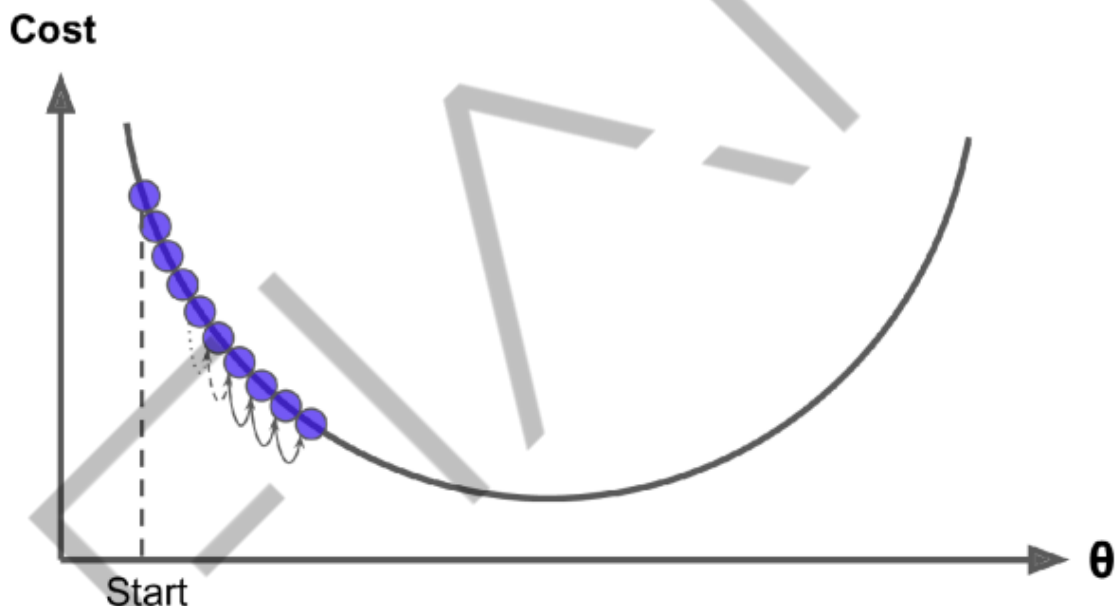


Figura 3 - Taxa de aprendizagem muito baixa
Fonte: Géron (2019)

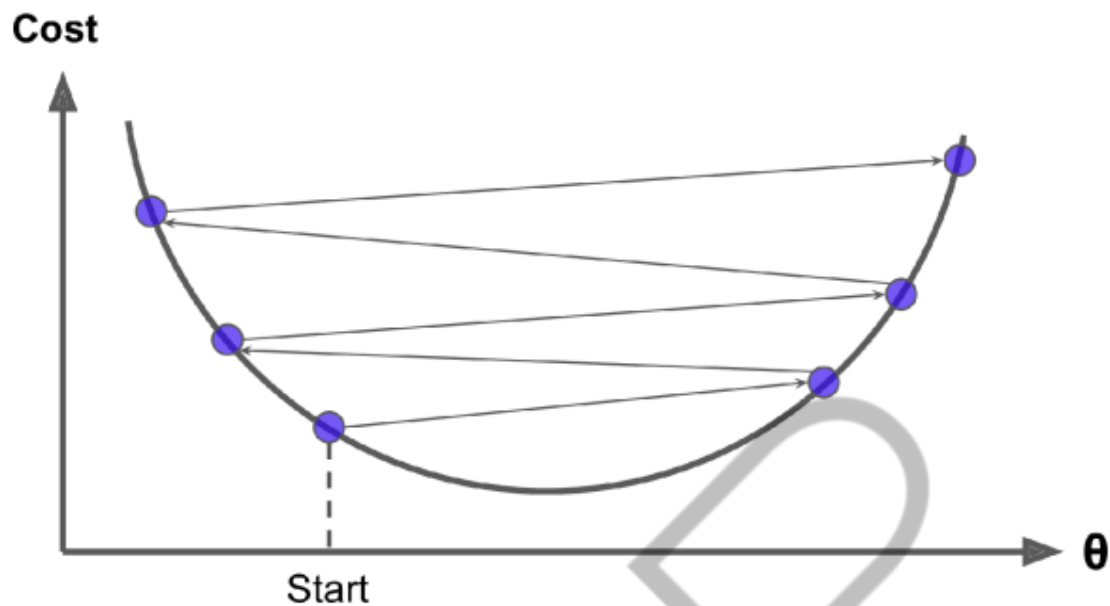


Figura 4 - Taxa de aprendizagem muito alta
Fonte: Géron (2019)

Observe na figura 3, que quando definimos os passos da taxa de aprendizagem como muito baixos, o algoritmo pode demorar muito para chegar no melhor mínimo local da função de custo devido aos passos lentos. Em contrapartida, na figura 4, com passos altos definidos por uma alta taxa de aprendizagem, a função de custo pode tornar o algoritmo divergente com valores cada vez maiores, não encontrando a melhor solução para o problema.

Como você pode observar nas figuras 3 e 4, a função de custo está sendo representada por tigelas regulares, mas nem sempre temos esse cenário. Pode haver muitos desafios na descida do gradiente (assim como a analogia com a montanha): buracos, planaltos e todo tipo irregular de terreno, fazendo com que a convergência ao melhor mínimo local seja muito difícil.

Quando temos esse cenário cheio de desafios o gradiente pode cair em uma “armadilha” tentando encontrar um melhor mínimo local na função de custo, sendo que nem sempre esse seria o melhor local. Como os valores são inicializados de forma aleatória, observe a figura 5.

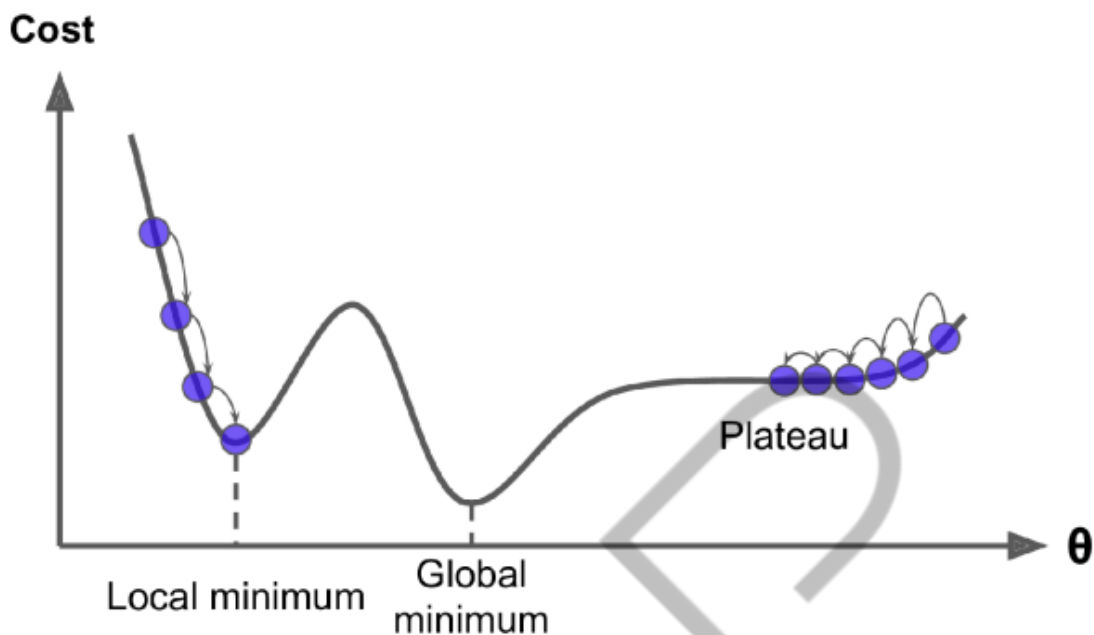


Figura 5 - Armadilha do gradiente descendente
Fonte: Géron (2019)

Imagine nesse cenário em que estamos longe de ter uma função de custo parecida com uma tigela regular se a inicialização aleatória do algoritmo iniciar ao lado esquerdo: ela converge para um mínimo local, o que não seria o ideal conforme podemos observar no melhor mínimo local global.

Se inicializar pela direita, poderá demorar muito para atravessar o planalto; se a função parar cedo demais, nunca chegará a um mínimo global. Quando pensamos em modelos de regressão linear, felizmente a métrica de função de custo (que pode ser a MSE) é uma função convexa, ou seja, com relação a quaisquer dois pontos da curva, o segmento de linha que os une nunca se cruza. Isso implica que não há mínimos locais e sim apenas um mínimo local, o que acaba nos livrando dessa armadilha do gradiente.

Escalonamento das features

A transformação das features em escalas padronizadas é essencial para a função de custo convergir no mínimo local de forma mais rápida e eficiente. Como você pode observar na figura 6, do lado esquerdo o algoritmo do gradiente descendente vai diretamente para o mínimo local, atingindo rapidamente, enquanto à direita o gradiente primeiro avança em sentido quase ortogonal em direção ao mínimo local e termina com uma longa marcha em um vale quase plano.

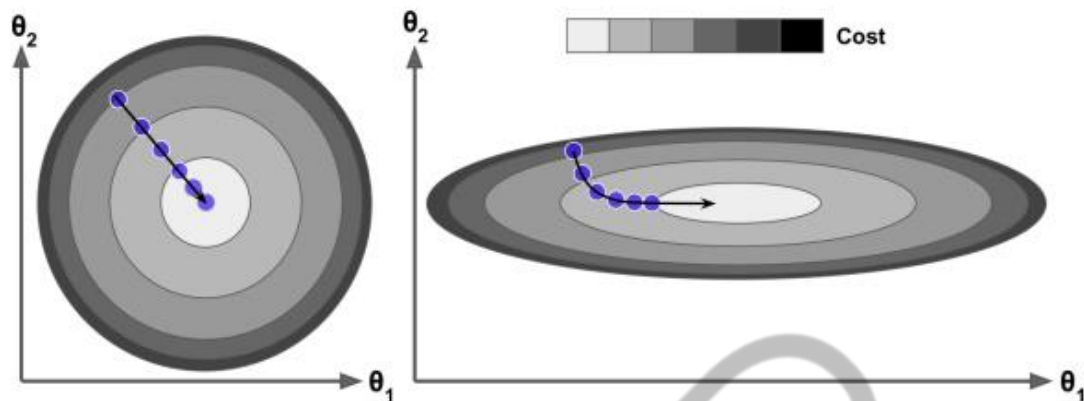


Figura 6 - Gradiente descendente com e sem escalonamento
Fonte: Géron (2019)

Para escalonar as features, você pode utilizar por exemplo o **Standard Scaler** do Sklearn. Dê uma olhadinha na biblioteca oficial [aqui](#),

Tipos de gradientes descendentes

Gradiente descendente em lote

Esse tipo de algoritmo utiliza todo o lote de dados em cada etapa para encontrar o melhor mínimo local. Como resultado, essa estratégia é terrivelmente lenta para grandes conjuntos de dados. No entanto, se dimensiona bem com a quantidade de características.

Quando pensamos em um modelo de regressão linear, quando há centenas de milhares de características, será muito mais rápido utilizar o algoritmo de gradiente do que o método dos mínimos quadrados.

Gradiente descendente estocástico (SGD)

Como você pode observar, o principal problema do gradiente descendente em lote é que ele utiliza todo o conjunto de dados de treinamento para calcular os gradientes em cada passo, o que torna o processo muito lento quando temos grandes datasets. No extremo oposto, o gradiente descendente estocástico escolhe uma instância aleatória no conjunto de treinamento em cada etapa e calcula os gradientes baseados apenas nessa única instância.

Isso torna esse tipo de gradiente bem mais rápido, visto que poucos dados são manipulados em cada iteração. Esta é uma ótima opção para grandes conjuntos de dados, uma vez que apenas uma instância precisa estar na memória a cada iteração.

Porém, devido a sua aleatoriedade (daí vem a palavrinha estocástica), esse algoritmo é bem menos regular que o gradiente descendente em lote. Em vez de diminuir suavemente até atingir o mínimo, a função de custo vai subir e descer, diminuindo apenas a média.

Ao longo do tempo ele acabará bem perto do mínimo, mas ao chegar lá o gradiente continua a rebater, nunca se estabilizando, gerando assim valores finais bons mas não ótimos.

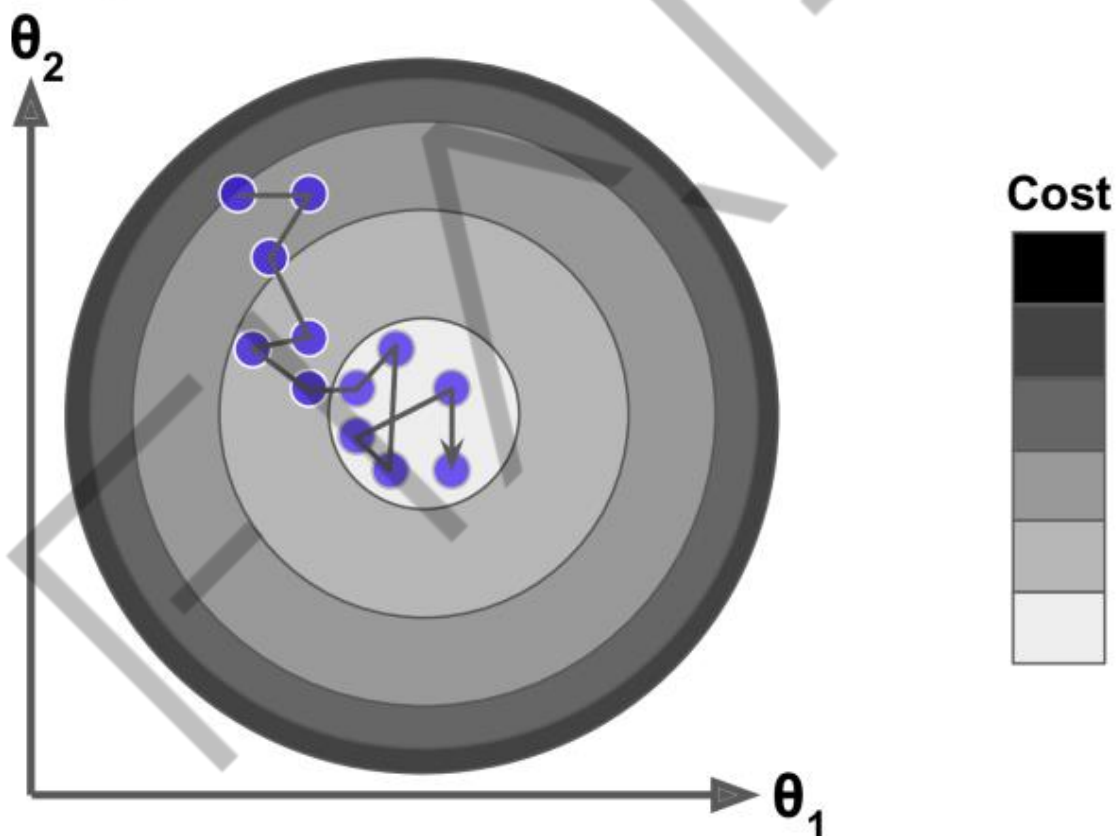


Figura 7 - Gradiente descendente estocástico
Fonte: Géron (2019)

Quando a função de custo é muito irregular, é possível ajudar o algoritmo a pular fora do mínimo local de modo que o gradiente estocástico tenha uma chance maior de encontrar o mínimo global do que o gradiente em lote. Desta forma, a

aleatoriedade é boa para escapar de um ótimo local, mas ruim porque significa que o algoritmo nunca pode se estabelecer no mínimo.

Uma boa solução para escapar dessa situação é diminuir gradualmente a taxa de aprendizagem. As etapas começam grandes, auxiliando a escapar dos mínimos locais, e depois diminuí-las, permitindo que o algoritmo se estabeleça no mínimo global.

Veja um exemplo de aplicação do gradiente estocástico com 10 iterações.

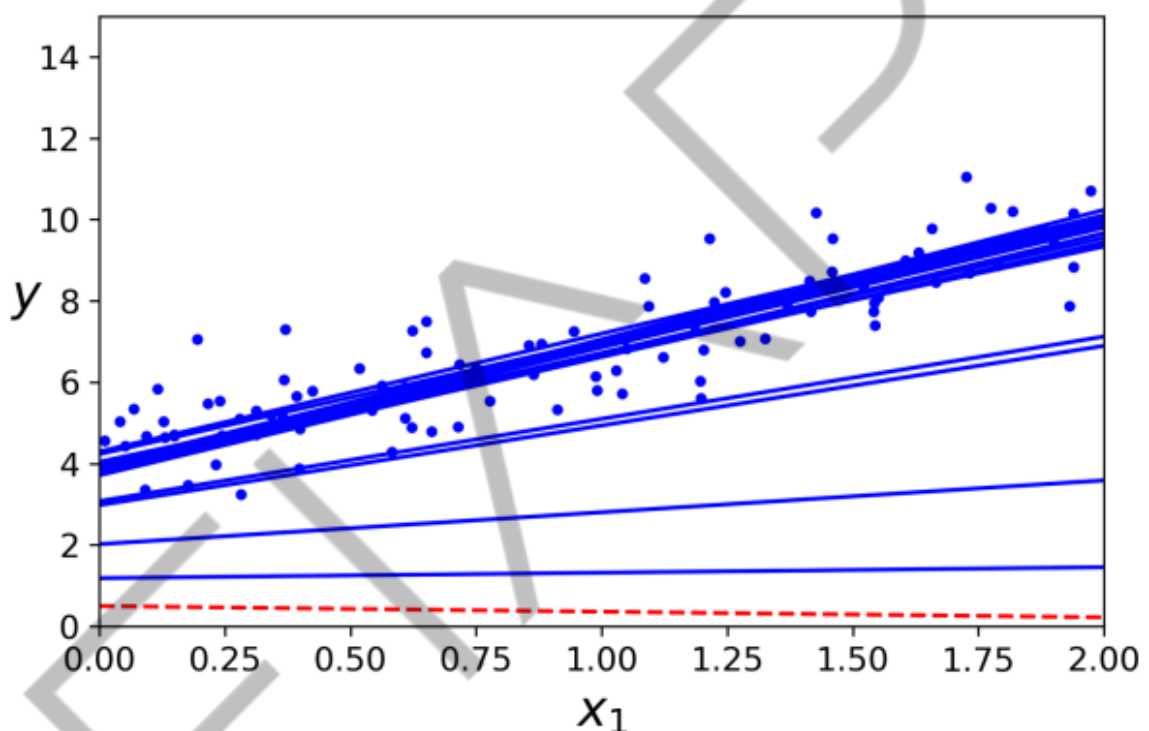


Figura 8 - Gradiente descendente estocástico sobre 10 iterações
Fonte: Géron (2019)

Essas iterações são chamadas de **épocas de processamento**. O parâmetro de iteração do gradiente é utilizado em conjunto com a escolha da taxa de aprendizagem. A figura 8 mostra a iteração de 10 épocas de processamento, observe que até a linha em azul chegar próxima das bolinhas em azul foram necessárias 10 iterações do gradiente para minimizar o erro da função de custo.

A seguir, temos um código do uso do tipo de gradiente estocástico empregando o modelo de regressão linear com SGD.

```
# Criando um dataset para teste
```

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


# Definir a faixa de temperatura e vendas

temperatura_min = 15

temperatura_max = 35

vendas_min = 50

vendas_max = 200


# Gerar dados aleatórios de temperatura

num_amostras = 500 # Número de amostras a serem geradas

temperaturas = np.random.rand(num_amostras) * (temperatura_max -
temperatura_min) + temperatura_min


# Calcular as vendas de sorvete (função linear)

def vendas_sorvete(temperatura):

    m = 2 # Inclinação da reta de regressão

    b = 100 # Intercepto da reta de regressão

    return m * temperatura + b

vendas_sorvete = vendas_sorvete(temperaturas)


# Adicionar ruído aleatório às vendas

vendas_sorvete += np.random.randn(num_amostras) * 10


# Criar o DataFrame

dados = pd.DataFrame({

    "Temperatura": temperaturas,

    "Vendas_Sorvete": vendas_sorvete

})
```

```
# Visualizar a distribuição dos dados

plt.scatter(dados["Temperatura"], dados["Vendas_Sorvete"])

plt.xlabel("Temperatura")

plt.ylabel("Vendas de Sorvete")

plt.title("Distribuição dos Dados")

plt.show()
```

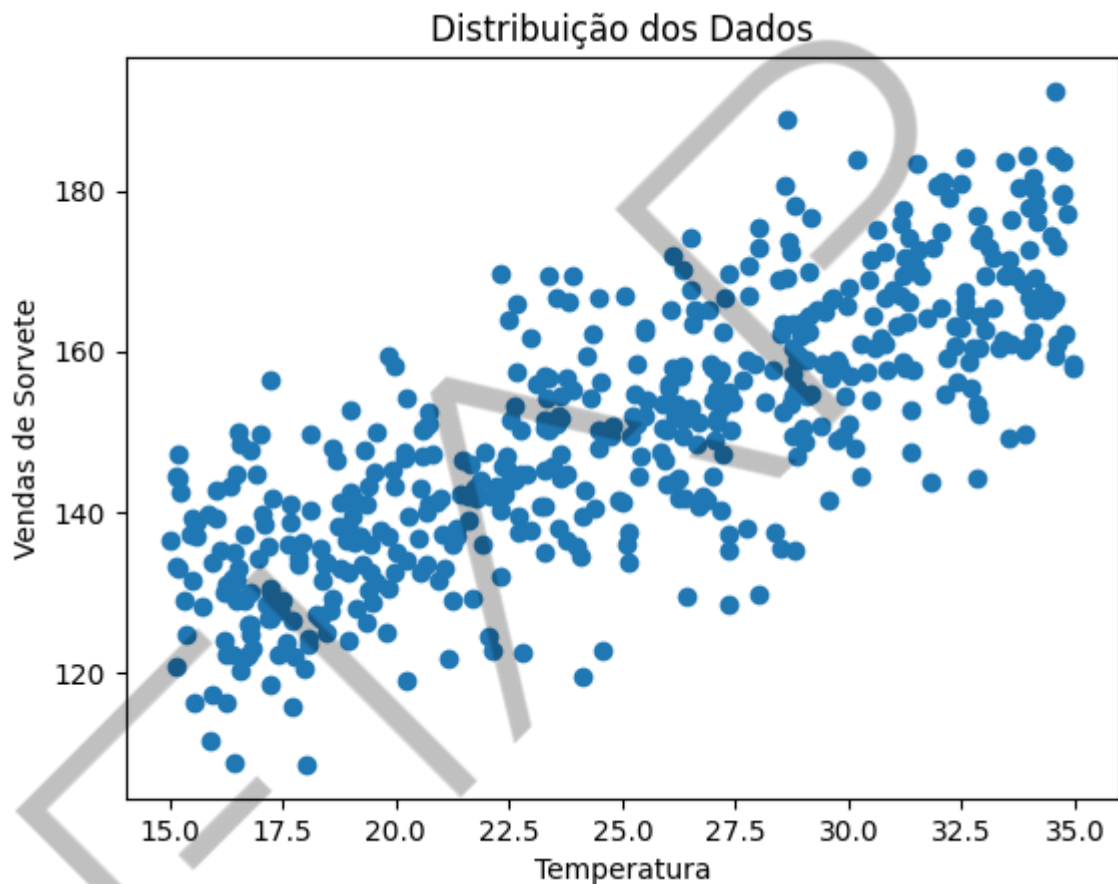


Figura 9 - Distribuição dos dados de vendas de sorvete por temperatura
Fonte: Criada pela autora (2024)

```
import numpy as np

import pandas as pd

from sklearn.linear_model import SGDRegressor

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

# Separar as features (Temperatura) e labels (Vendas_Sorvetes)
```

```
X = dados["Temperatura"].values.reshape(-1, 1)
y = dados["Vendas_Sorvete"].values

# Normalizar as features (Temperatura)
scaler = StandardScaler()
X_norm = scaler.fit_transform(X)

# Treinar o modelo com SGDRegressor e hiperparâmetros ajustados
sgd_reg = SGDRegressor(max_iter=1000, penalty='l2', eta0=0.01)
sgd_reg.fit(X_norm, y.ravel())

# Analisar os resultados
print("Inclinação (m):", sgd_reg.coef_[0])
print("Intercepto (b):", sgd_reg.intercept_)

# Fazer previsões para novos valores de temperatura
nova_temperatura = 22.5

# Reshape nova_temperatura em um 2D array com linha e coluna
nova_temperatura = np.array([nova_temperatura]).reshape(1, 1)

# Aplicar o scaler em temperatura
nova_temperatura_scaled = scaler.transform(nova_temperatura)

# Nova previsão utilizando o scaled nova_temperatura
predicao = sgd_reg.predict(nova_temperatura_scaled)
print("A previsão de vendas para a temperatura de 22.5 graus é de",
predicao)
```

```
# Calcular o R² e outras métricas

from sklearn.metrics import r2_score, mean_squared_error,
mean_absolute_error

# Criando a função do MAPE
def mean_absolute_percentage_error(y_true, y_pred):
    y_true = np.clip(y_true, np.min(y_pred), np.max(y_pred))
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100

r2 = r2_score(y, sgd_reg.predict(X_norm))
mse = mean_squared_error(y, sgd_reg.predict(X_norm))
rmse = np.sqrt(mse)
mae = mean_absolute_error(y, sgd_reg.predict(X_norm))
mape = mean_absolute_percentage_error(y, sgd_reg.predict(X_norm))

print("R²:", r2)
print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("MAPE:", mape)

# Visualizar a relação entre features e labels com reta de regressão
plt.scatter(X, y, label="Dados Originais")
plt.plot(X, sgd_reg.predict(X_norm), color="red", label="Reta de Regressão")
plt.xlabel("Temperatura")
plt.ylabel("Vendas de Sorvete")
plt.title("Modelo de Regressão Linear")
plt.legend()
plt.show()
```


Analizando os resultados:

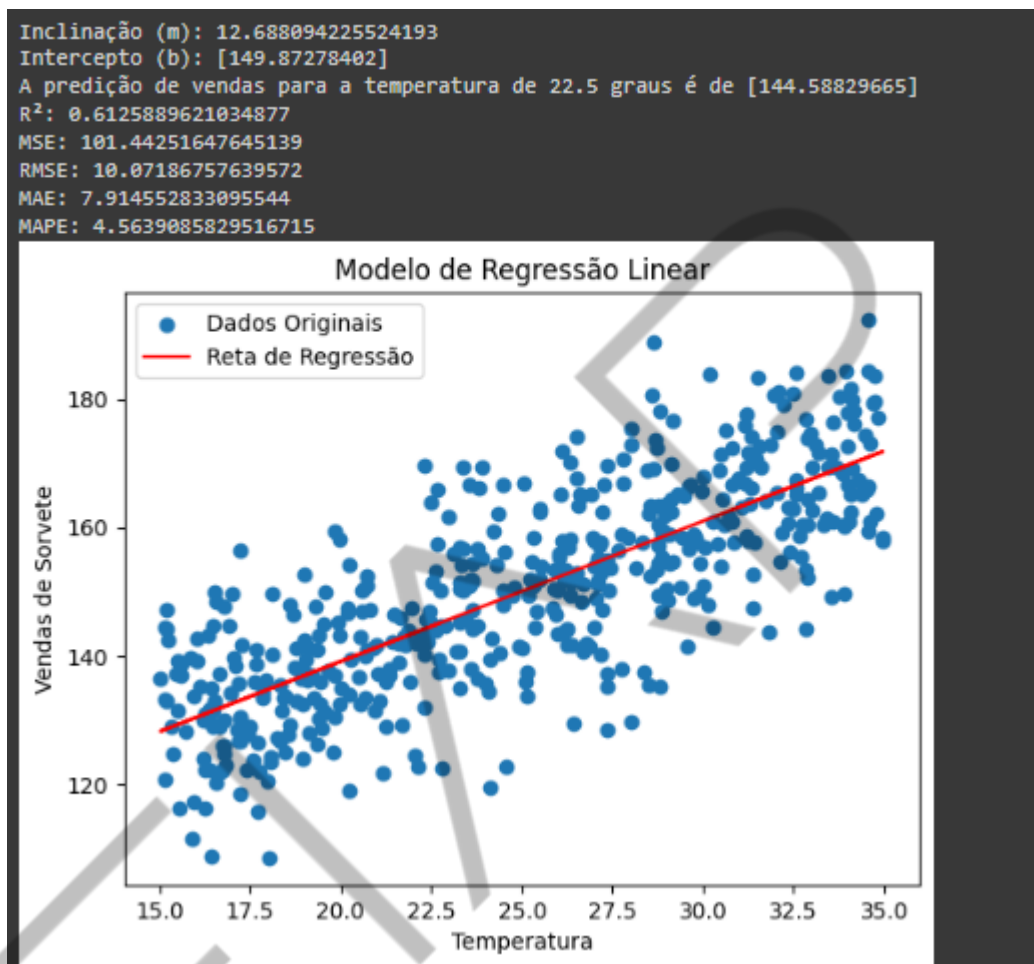


Figura 10 - Resultado da predição com o SGDRegressor
Fonte: Elaborado pela autora (2024)

Esse foi apenas um exemplo da aplicação do gradiente em modelo de regressão. Nas videoaulas, você vai aprender como utilizar mais algoritmos de gradiente descendente.

Gradiente descendente em mini lotes

Nesse tipo de algoritmo, em vez de calcular os gradientes a cada etapa com base em um conjunto de dados completo de treinamento (como o gradiente descendente em lote) ou em apenas uma instância (como o SGD), o gradiente em mini lotes faz o cálculo dos gradientes em pequenos conjuntos aleatórios de instâncias chamados de mini lotes.

A principal vantagem aqui é que você pode obter ganho de desempenho na otimização de hardware das operações de matriz, especialmente quando utilizamos GPUs.

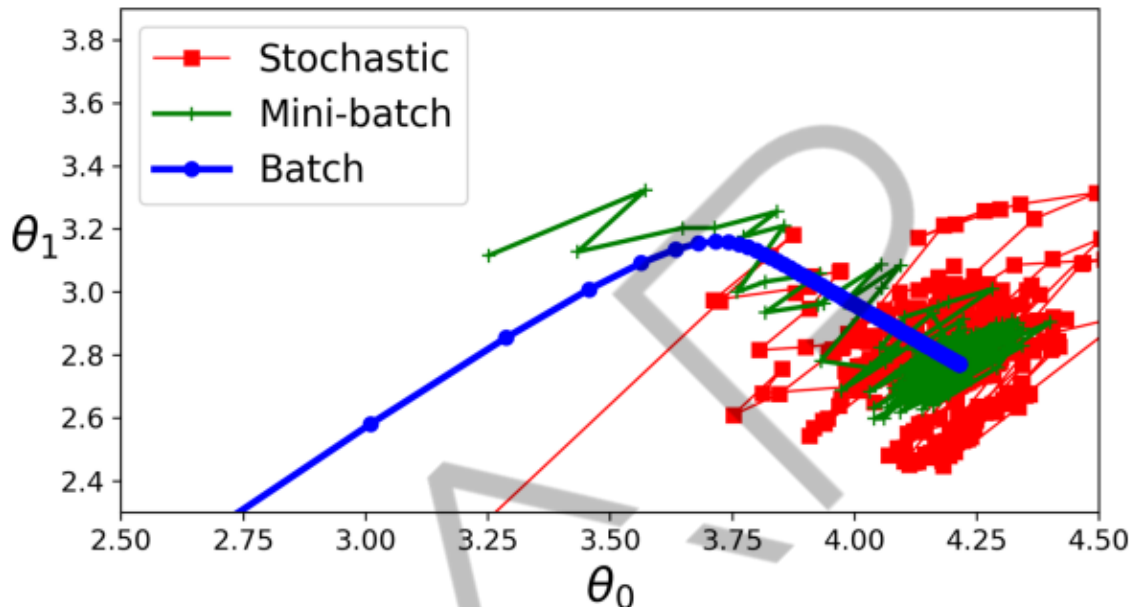


Figura 11 - Comparando os 3 tipos de gradientes
Fonte: Géron (2019)

Descida de Gradiente com Momento

Exemplificando de forma bem simples, podemos comparar esses tipos de otimizador com uma bola de boliche rolando em um declive suave em uma superfície lisa (sua rolagem iniciará de forma lenta, porém logo pegará impulso até que chegue à velocidade final).

A otimização Momentum se preocupa muito com os gradientes anteriores: a cada iteração, ela subtrai o gradiente local do vetor momentum m (multiplicado pela taxa de aprendizado η) e atualiza os pesos adicionando este vetor momentum. O algoritmo adiciona um novo hiperparâmetro β (momentum) que deve ser ajustado entre 0 até 1. Um bom e típico valor para esse hiperparâmetro costuma ser 0,9.

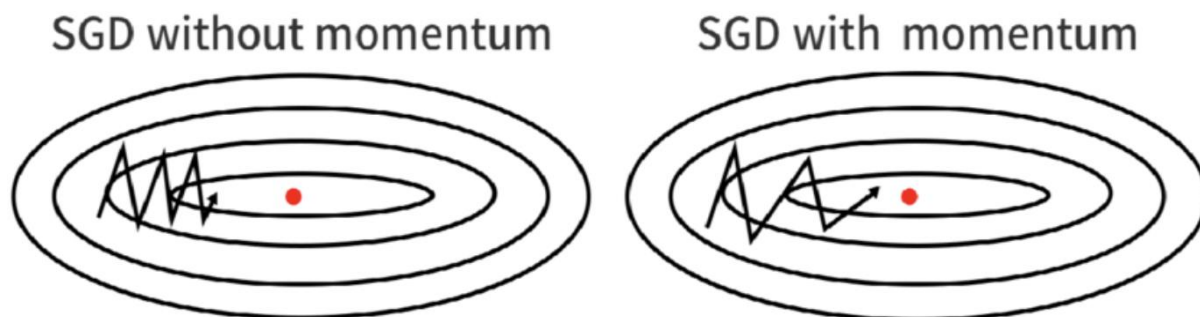


Figura 12 - SGD sendo utilizado com e sem momentum
 Fonte: Papers with Code [s.d.]

Gradiente Acelerado de Nesterov

Podemos definir o Gradiente Acelerado Nesterov como uma variação do algoritmo de momentum, em que quase sempre essa opção é mais rápida do que a otimização de momentum normal. A ideia do algoritmo é medir o gradiente da função de custo não na posição local, mas ligeiramente à frente, na direção do momentum. Em comparação com a otimização de momentum, o Nesterov quase sempre acelerará o treinamento.

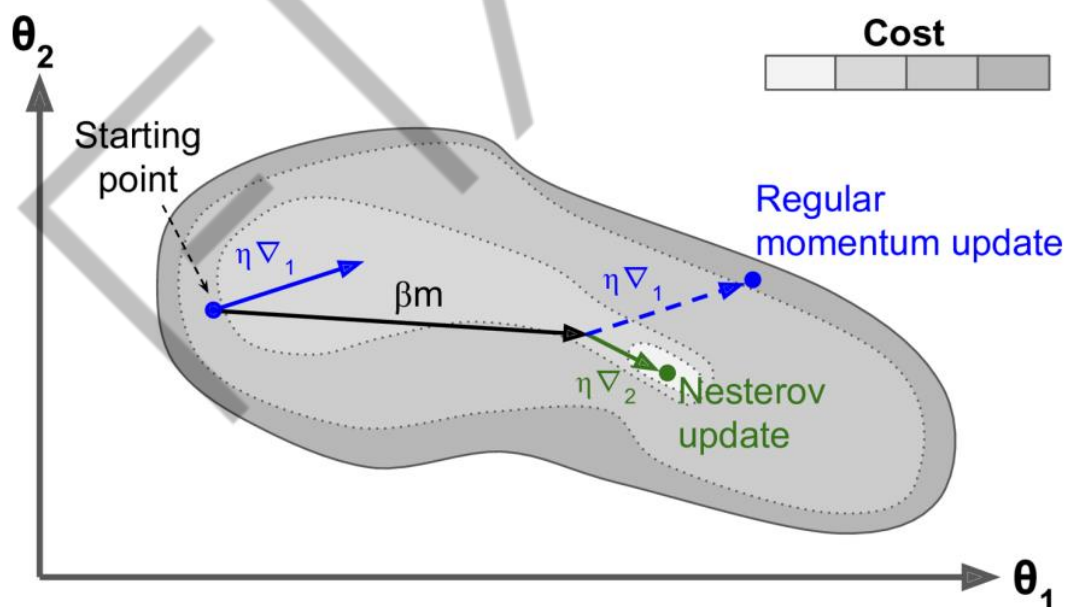


Figura 13 - Otimizador Nesterov
 Fonte: Géron (2019)

Gradiente descendente AdaGard

Considere o seguinte exemplo: imagine uma tigela alongada em que o Gradiente Descendente começa a descer a encosta mais íngreme rapidamente para depois descer mais lentamente em direção ao fundo. Seria bom o algoritmo detectar isso rapidamente e corrigir sua direção para direcionar mais para o global optimum (melhor mínimo local). O algoritmo AdaGrad corrige essa situação, ajustando automaticamente o passo de aprendizagem para cada parâmetro considerando o histórico de gradientes acumulados.

Basicamente o AdaGrad escala o vetor gradiente na dimensão mais íngreme. Ele geralmente funciona bem para problemas quadráticos simples, mas infelizmente muitas vezes para muito cedo de treinar quando se trata de problemas mais complexos como as redes neurais. A taxa de aprendizagem diminui tanto que o algoritmo acaba parando completamente antes de encontrar um melhor mínimo local.

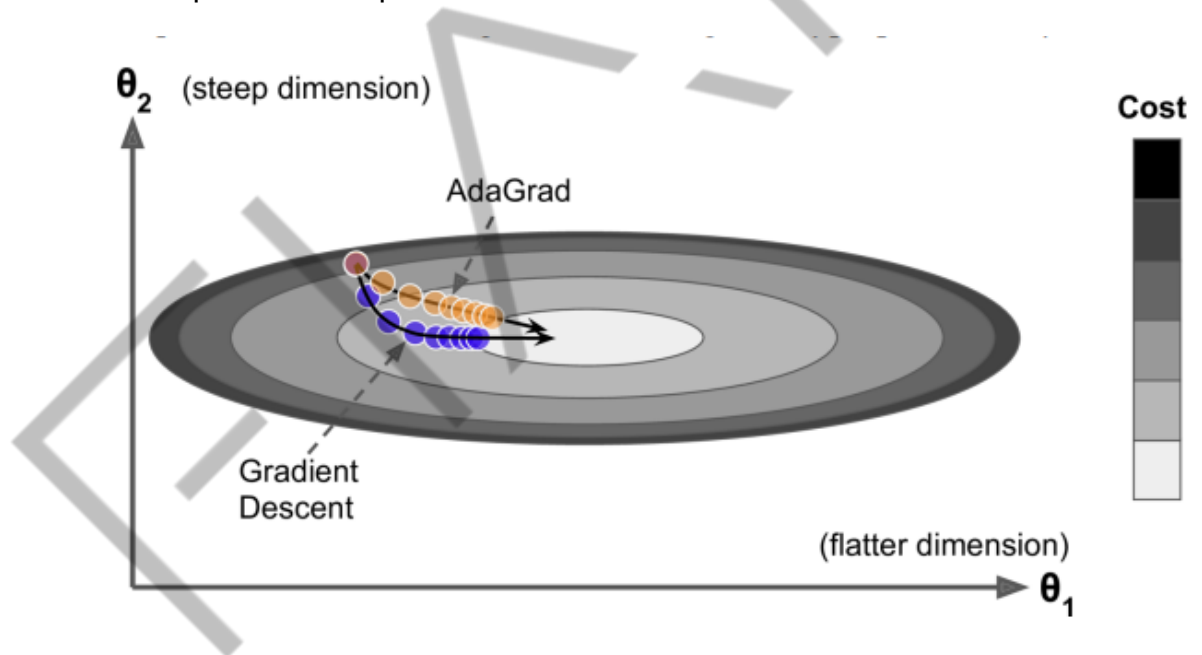


Figura 14 - Otimizador AdaGrad
Fonte: Géron (2019)

RMSProp

O otimizador RMSProp corrige o que o AdaGrad faz: o algoritmo **acumula somente os gradientes das interações mais recentes** (em vez de todos os gradientes desde o início do treinamento) por meio do decaimento exponencial no primeiro passo.

A taxa de degradação é ajustada para 0,9; geralmente esse valor padrão funciona bem. Exceto para problemas muito simples, o otimizador RMSProp quase sempre funciona muito bem e melhor que o AdaGrad. Esse otimizador já foi um dos favoritos da comunidade, mas perdeu seu posto para o famosinho Adam.

Otimização Adam

O Adaptive Moment Estimation (Adam) combina as ideias do Momentum e do RMSProp: assim como a otimização de momentum, o Adam acompanha uma média exponencialmente decadente de gradientes passados, e assim como o RMSProp controla uma média exponencialmente decadente de gradientes quadrados passados. O otimizador Adam requer menos configuração de hiperparâmetro da taxa de aprendizagem, então um valor padrão $\eta = 0,001$ pode ser uma boa opção!

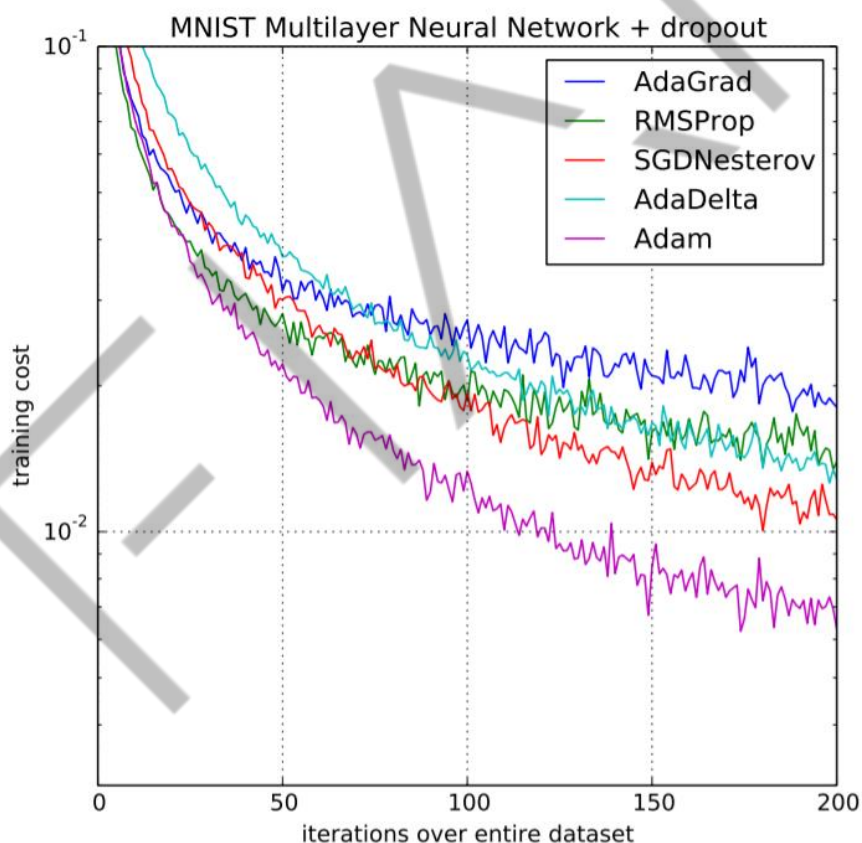


Figura 15 - Comparação do otimizar Adam com outros tipos de algoritmos
Fonte: Acervo Lima (2022)

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu sobre os tipos mais famosos de gradiente e suas variações. Lembrando que algoritmos de gradiente são ferramentas essenciais para solucionar problemas de otimização em diversas áreas.

A escolha do algoritmo ideal depende da natureza da função, dos recursos computacionais disponíveis e do nível de precisão desejado.

EMEND

REFERÊNCIAS

ACERVO LIMA. **Intuição de Adam Otimizador**. 2022. Disponível em: <<https://acervolima.com/intuicao-de-adam-otimizador/>>. Acesso em: 17 jun. 2024.

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. 2. ed. [s.l.]: O'Reilly Media, Inc., 2019.

PAPERS WITH CODE. **SGD with Momentum**. [s.d.]. Disponível em: <<https://paperswithcode.com/method/sgd-with-momentum>>. Acesso em: 17 jun. 2024.

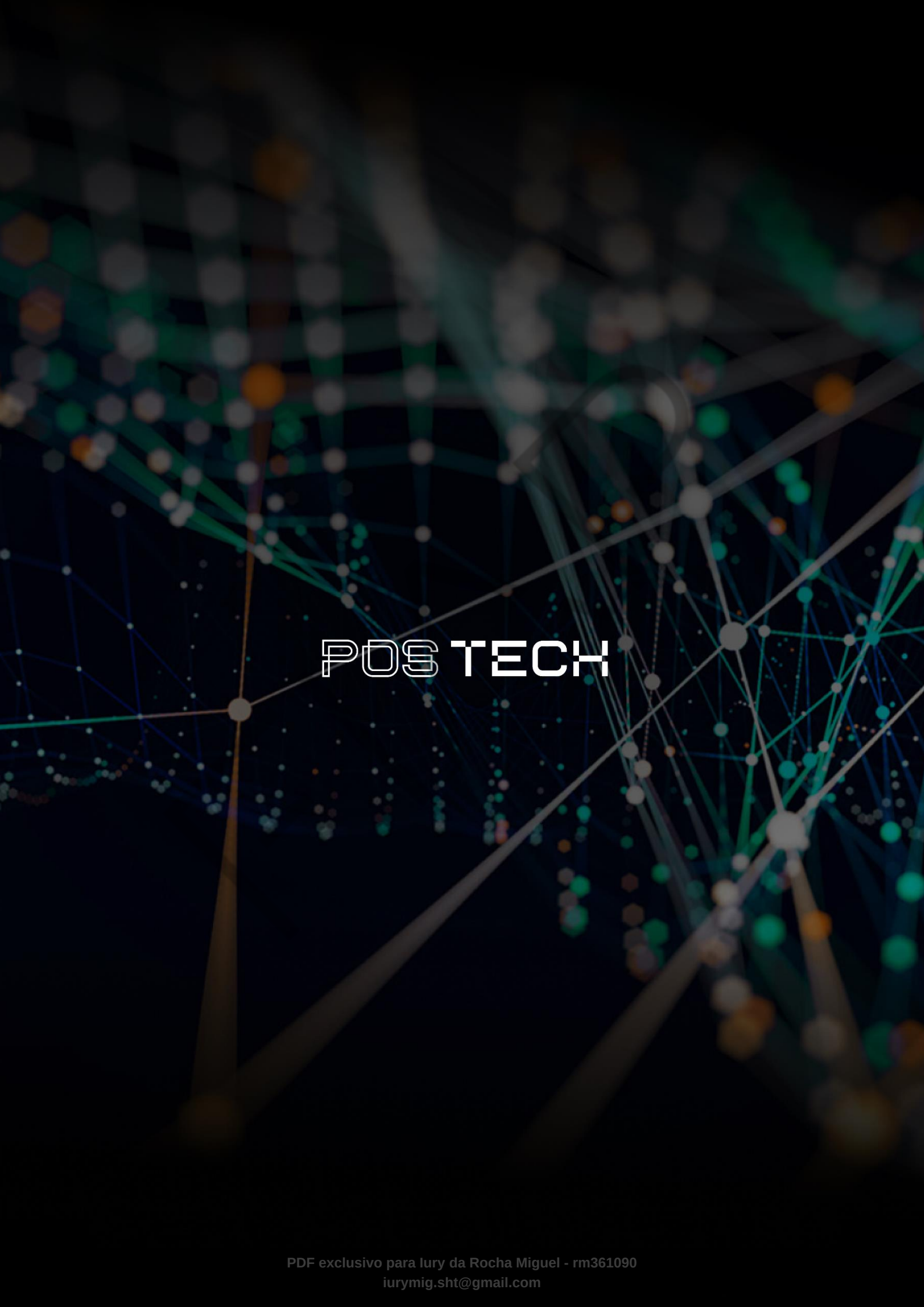
SCIKIT LEARN. **StandardScaler**. 2024. Disponível em: <<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>>. Acesso em: 17 jun. 2024.

VAZ, A. L. **Gradientes Descendentes na prática — melhor jeito de entender**. 2020. Disponível em: <<https://medium.com/data-hackers/gradientes-descendentes-na-pratica-melhor-jeito-de-entender-740ef4ff6c43>>. Acesso em: 17 jun. 2024.

PALAVRAS-CHAVE

Palavras-chave: Gradiente Descendente. Otimização. Taxa de aprendizagem.

EMSE



POSTECH